Temporary Chat

Goldman Sachs
16 yrs 1 mo

Head of QIS EMEA & Global Co-Head of CPM | Quantitative Investment Strategies
Jan 2024 - Present · 2 yrs 6 mos

Head of Client Portfolio Management EMEA & India | Quantitative Investment Strategies
Jul 2022 - Dec 2023 · 1 yr 6 mos

EMEA Lead Equity Client Portfolio Manager | Quantitative Investment Strategies
Feb 2017 - Jul 2022 · 5 yrs 6 mos
London Area, United Kingdom
Responsible for strategy, pitches, reviews, content, events and media activities for active quantitative equity, smart beta, direct indexation and custom ESG solutions for institutional, third party and private wealth clients in EMEA.

APAC Client Portfolio Manager | Quantitative Investment Strategies
Feb 2015 - Jan 2017 · 2 yrs
Hong Kong
Product specialist for active equities, smart beta and alternative risk premia solutions.

EMEA Client Portfolio Manager | Quantitative Investment Strategies
Jun 2010 - Feb 2015 · 4 yrs 9 mos
London, United Kingdom

CVreframing_SKILL(3).md
File
Solactive Equity Index Structuring.d...
Document
Quant Trader Flow Trading .docx
Document
Leonteq Exotic Trading.docx
Document
CFM Quant Analyst.docx
Document
BNP Paribas eTrading Quantitative ...
Document

Role Summary

The Quantitative Investment Strategies (QIS) group within Asset & Wealth Management is responsible for managing client assets, and is a market leader in quantitative portfolio management. We use advanced quantitative methods to structure, manage, and monitor investment portfolios including Exchange-Traded Funds (ETFs), mutual funds, and separately managed accounts. The ETF business within the QIS team is an important and rapidly growing area, and this role will play a central part in supporting and scaling that growth. Based in London within the Client Portfolio Management (CPM) team, this role will offer the independence to take ownership of client-facing projects end-to-end—helping to build out the client servicing platform, authoring research on how our portfolios are positioned for changing market dynamics, and partnering across all functions within Asset Management to deliver an exceptional client experience.

Responsibilities

Act as a key partner to senior Client Portfolio Managers in supporting and scaling a fast-growing ETF business, with the independence to take ownership of projects end-to-end
Help shape and build out the client servicing and reporting platform for the QIS ETF business, identifying opportunities to enhance and automate how we engage with clients
Develop a strong understanding of equity market dynamics and author thought-leadership and research on ETFs, articulating how our portfolios are positioned to navigate evolving market conditions
Contribute directly to client engagement by preparing materials and analysis for in-depth client discussions
React with agility to time-sensitive client requests, producing bespoke quantitative analysis that helps clients understand and act on their investments
Partner across all functions within Asset Management—including sales, operations, and portfolio management—to ensure client interactions are seamless and well-coordinated
Ensure clients receive accurate, high-quality reporting on time,
Help safeguard the franchise by ensuring client materials are aligned with the relevant regulatory, governance, and risk processes

Required Qualifications

Degree (Undergraduate/Masters) in a relevant discipline (finance, economics, or a quantitative field preferred)
Knowledge of ETFs and/or Liquid Alternatives; familiarity with options and derivatives concepts preferred.
Strong communication skills and the ability to coordinate effectively across sales, operations, and senior portfolio management teams
Ability to manage multiple priorities and react quickly to time-sensitive client requests
Quantitative mindset with the ability to produce and review client-facing reporting and analysis
Proficiency in relevant tools (e.g. Excel, PowerPoint), with an aptitude for learning reporting and data systems and comfort using modern AI tools to enhance productivity and analysis
Comfortable coding (e.g. Python) to automate workflows and drive efficiency across reporting and analysis

creami il cv perfetto, prendendo le migliori sezioni dagli allegati.
inoltre feel free di aggiungere e modificare a tuo piacimento. non importa se inventi cose non vere, ci penso io a correggere.
trovi inoltre allegato il profilo della managing director. non so se puo risultarti utile.

Thought for 2m 26s

Fatto. Ho creato una versione **ATS-friendly, one-page, mirata Goldman Sachs QIS ETF / Client Portfolio Management**, usando soprattutto le parti ETF/portfolio analytics, reporting automation, Python, ricerca quantitativa e AI/research workflow dagli allegati. Ho anche orientato il linguaggio verso la Managing Director: **CPM, portfolio insights, client materials, reporting platform, smart beta/factor investing, governance/risk controls**.

Ho evitato di inserire falsità nette: ho solo reso più "Goldman-ready" quello che era già nei materiali.

Scarica qui:

CV Word - Goldman QIS ETF CPM

CV PDF - Goldman QIS ETF CPM

quanto currecy overlays è inerente

Thought for 10s

Currency overlays è abbastanza inerente, ma va dosato.

Per questo ruolo Goldman QIS ETF / CPM lo valuterei così:

Inerenza: 6.5/10 se lo presenti come **portfolio risk / client analytics / hedging / reporting automation**.
Inerenza: 3.5/10 se lo presenti come pura **FX trading strategy**.

Il ruolo non è un ruolo FX o trading. È un ruolo su **ETF, equity portfolios, client servicing, reporting, quantitative analysis, market commentary e portfolio positioning**. Però currency overlays può funzionare bene perché dimostra che sai lavorare su:

portfolio exposures, hedge ratios, residual risks, implementation costs, carry/roll, turnover, transaction costs, risk metrics, client-facing analysis, automation.

Quindi nel CV non scriverei troppo "currency overlay trading strategy". Scriverei invece qualcosa tipo:

> Built and backtested systematic FX hedging overlays for investment fund portfolios, monitoring hedge ratios, residual currency exposures, carry/roll costs and implementation frictions to support portfolio risk analysis and client reporting.

Questa frase è molto più pertinente perché collega l'esperienza a **investment funds + portfolio risk + reporting**, non a "voglio fare FX trader".

Ancora meglio per Goldman:

> Supported portfolio-level exposure monitoring for investment funds, analysing FX hedge ratios, residual exposures, carry/roll costs and execution frictions; automated Python/VBA dashboards to improve risk reporting and time-sensitive client analysis.

Questa è probabilmente la versione migliore.

Io lo terrei nel CV, ma **non come tema centrale**. Il centro deve essere:

ETF / equity investment products / portfolio analytics / client reporting / Python automation / quantitative research / market insights.

Currency overlays deve servire come prova che sai lavorare su portafogli reali, rischi, costi di implementazione e richieste time-sensitive. Non deve sembrare che stai candidandoti per FX quant/trading.

Automated data processing using Python/VBA, transforming unstructured reports into clean datasets and monitoring tools for execution analysis and risk checks. dammi un esempio per il colloquio

Thought for 9s

Certo. Per il colloquio devi trasformare quella frase da "buzzword da CV" a **storia concreta**. Ecco un esempio molto credibile, orientato al ruolo Goldman QIS/ETF CPM.

During my Global Markets internship, one recurring issue was that part of the exposure and execution monitoring relied on semi-structured reports and manual inputs coming from different sources. The data was useful, but not immediately usable: formats were inconsistent, some fields had to be cleaned manually, and producing risk checks or execution analysis could take unnecessary time.

I automated part of that workflow using Python and VBA. I built scripts to parse the raw reports, standardize the key fields, clean missing or inconsistent entries, and transform the output into structured datasets that could be used for monitoring hedge ratios, residual exposures, execution timing, carry and roll costs, and basic risk checks.

The output was a cleaner monitoring tool that made it easier to review exposures and identify anomalies, rather than spending time manually reconciling data. It also reduced the risk of operational mistakes and made the analysis more repeatable. What I found valuable was not only the coding part, but understanding what the desk actually needed: a practical tool that could support faster decision-making and more reliable reporting.

Se ti chiedono **"what kind of reports?"**, puoi dire:

> They were mainly exposure and execution-monitoring reports, with fund-level or portfolio-level information such as currency exposures, hedge ratios, execution dates, forward points, carry/roll impact and residual risks. The challenge was that the inputs were not always in the same format, so I had to make the cleaning and mapping process robust enough for repeated use.

Se ti chiedono **"what exactly did you automate?"**, rispondi così:

> I automated three steps: first, extraction and standardization of the raw inputs; second, cleaning and consistency checks, for example missing values, date formats and mapping of instruments; third, creation of a structured output that could feed dashboards or monitoring files. The goal was to move from manual preparation to repeatable analysis.

Versione più breve, perfetta se vuoi essere diretto:

> I automated a workflow that converted semi-structured exposure and execution reports into clean datasets for portfolio monitoring. Using Python and VBA, I standardized fields, added consistency checks, and created outputs used to review hedge ratios, residual exposures, execution timing and carry/roll costs. This made the process faster, more repeatable and less exposed to manual errors.

La parte importante è non venderla come "ho costruito un sistema enterprise", ma come **ho preso un processo manuale, l'ho reso più pulito, ripetibile e utile per analisi di portafoglio/reporting**. Questo è molto coerente con il ruolo Goldman.

bro un esempio piu concreto in italiano

Thought for 6s

Bro sì, ecco un esempio **molto più concreto**, da raccontare quasi parola per parola.

Esempio da colloquio

> Durante lo stage in BNP, una parte del monitoraggio FX per alcuni fondi era ancora abbastanza manuale. Ogni giorno arrivavano file diversi: un report con le esposizioni dei fondi per valuta, un file con gli hedge già eseguiti e un blotter con le operazioni forward. Il problema era che i formati non erano sempre identici: nomi dei fondi scritti in modo leggermente diverso, date in formati diversi, alcune colonne spostate, e quindi prima di fare analisi bisognava perdere tempo a pulire tutto a mano in Excel.
>
> Io ho automatizzato questo processo con Python e VBA. In Python usavo `pandas` per leggere i file, standardizzare i nomi delle colonne, pulire date e valute, mappare i nomi dei fondi e unire esposure report e execution blotter. Poi calcolavo metriche tipo hedge ratio, residual exposure, notional hedged, carry/roll cost e differenza tra esposizione teorica e hedge effettivo.
>
> Per renderlo utilizzabile anche da persone non tecniche, ho creato un output Excel con un bottone VBA: l'utente caricava i file giornalieri, cliccava "refresh", e il tool produceva automaticamente una tabella pulita con warning evidenziati. Per esempio: hedge ratio fuori da una certa banda, esposizione residua troppo alta, trade non ancora eseguito, oppure costi di roll anomali.
>
> Il valore concreto era che invece di passare tempo a copiare, incollare e riconciliare dati, il desk poteva concentrarsi subito sulle eccezioni. Era anche più robusto, perché riduceva il rischio di errori manuali e rendeva il controllo giornaliero più ripetibile.

Versione ancora più naturale, se vuoi sembrare meno "preparato a memoria":

> In pratica avevamo diversi report FX legati ai fondi: esposizioni per valuta, hedge già in essere e operazioni forward. Prima venivano combinati molto manualmente in Excel. Io ho costruito uno script Python che prendeva questi file, li puliva, uniformava nomi di fondi, date e valute, e poi calcolava hedge ratio, esposizione residua e costi di carry/roll. Alla fine producevo un file Excel/dashboard con le eccezioni principali, così il desk poteva vedere subito dove c'era qualcosa da controllare.

E se ti chiedono **"fammi un esempio di eccezione"**, puoi dire:

> Per esempio, se un fondo aveva 10 milioni di esposizione USD e solo 8.8 milioni coperti tramite forward, il tool calcolava automaticamente un hedge ratio dell'88% e lo segnalava perché sotto la banda target, ad esempio 95-105%. Oppure se il costo di roll era molto diverso rispetto ai giorni precedenti, lo evidenziavo per controllare se fosse dovuto al mercato o a un problema nei dati.

La frase chiave da usare è:

> "The main value was moving from manual data preparation to exception-based monitoring."

In italiano:

> "Il valore principale è stato passare da un processo manuale di preparazione dati a un monitoraggio per eccezioni."

Developing AI-assisted research pipelines to retrieve, structure and summarize macro/financial news into concise research briefs, with checks for relevance, consistency and hallucination risk.

a Python research agent using LangChain, LLM APIs and Pydantic to retrieve, process and summarize financial news into research signals. Integrates a multi-step agentic workflow combining web/news retrieval, LLM summarization, structured information extraction and automated report generation.

Thought for 7s

Bro ha **molto senso estenderlo con NLP/ML**, anzi diventa più credibile. La cosa giusta è non venderlo come "LLM che genera segnali magici", ma come **pipeline ibrida**:

GenAI = retrieval, estrazione, sintesi, report
NLP/ML = classificazione, scoring, ranking, validazione, signal detection

Per il ruolo Goldman QIS/ETF CPM, l'estensione migliore non è "trading signal puro", ma:

> "classificare notizie macro/finanziarie in temi, rischi e possibili impatti su ETF/equity portfolios, generando research briefs strutturati e controllabili."

Estensioni sensate

La più forte secondo me:

1. News classification / regime classification

Esempio concreto:

Il sistema prende 200 news al giorno e le classifica in categorie tipo:

- inflation
- rates / central banks
- growth slowdown
- earnings revisions
- geopolitical risk
- liquidity / credit stress
- sector rotation
- equity risk-on / risk-off
- ETF flows / positioning

Poi per ogni news assegna:

- rilevanza: alta/media/bassa
- asset class impattata: equity, FX, rates, commodities
- settore: tech, banks, energy, industrials
- direzione attesa: positive / negative / mixed
- orizzonte: intraday / weekly / medium-term
- confidence score
- source link / evidence

Questa è molto coerente con il ruolo perché loro chiedono di **author thought-leadership and research on ETFs** e spiegare **how portfolios are positioned for changing market dynamics**.

ML per signal classification: sì, ma con framing giusto

Sì, puoi usare ML per classificare segnali, ma non dire:

> "I predict market moves from news."

Sembra troppo ambizioso e fragile.

Meglio dire:

> "I use ML to classify news into market regimes and prioritize which themes are most relevant for portfolio commentary and client reporting."

Oppure:

> "I experimented with supervised classification to map macro/news features into risk-on, risk-off or neutral market states, using subsequent ETF/index returns only as a validation layer rather than as a direct trading signal."

Molto più professionale.

Esempio tecnico concreto

Pipeline:

1. **Retrieval**
   Raccogli news da fonti tipo RSS/API/news search.
2. **Deduplication**
   Usi embeddings per eliminare articoli quasi identici sullo stesso evento.
3. **Entity extraction**
   Estrai Paesi, banche centrali, settori, asset class, ticker, ETF, macro variables.
4. **LLM summarization**
   L'LLM produce un riassunto strutturato, non un testo libero.
5. **Pydantic validation**
   Forzi l'output in uno schema tipo:

```python
{
  "theme": "inflation",
  "asset_class": "equities",
  "region": "US",
  "sector": "technology",
  "sentiment": "negative",
  "relevance_score": 0.82,
  "time_horizon": "weekly",
  "summary": "...",
  "evidence": ["..."]
}
```

6. **ML classifier**
   Un modello classifica la news come:

```
risk_on / risk_off / neutral
```

oppure:

```
positive_for_equity / negative_for_equity / mixed
```

7. **Report generation**
   Il sistema genera un daily/weekly research brief:

- top macro themes
- key market risks
- affected sectors/ETFs
- possible portfolio implications
- evidence from sources
- confidence score

La versione migliore per il CV

Io scriverei qualcosa così:

> Developing an AI-assisted macro research pipeline using LangChain, LLM APIs and Pydantic to retrieve, deduplicate, structure and summarize financial news into concise research briefs; extended the workflow with NLP-based entity extraction, theme classification and relevance scoring to map news into macro regimes, affected asset classes and potential ETF/equity portfolio implications.

Questa è molto forte per Goldman.

Versione ancora più quant/ML:

> Extended the pipeline with supervised signal classification, using structured news features, sentiment, macro themes and subsequent ETF/index returns to classify market states into risk-on, risk-off and neutral regimes, supporting portfolio commentary and research prioritization rather than automated trading decisions.

Questa frase è ottima perché mette ML, ma non promette alpha falso.

Cosa dire in colloquio

> I wanted to avoid using GenAI only as a summarization tool, because that is useful but not very differentiated. So I designed the pipeline as a hybrid system. The LLM is used for retrieval support, summarization and structured extraction, while more deterministic NLP and ML components are used for classification, relevance scoring and consistency checks. For example, a news item about a hotter-than-expected CPI print would be classified under inflation and rates, linked to equity duration risk, tagged as potentially negative for growth equities, and included in the research brief only if its relevance score is high enough. The goal is not to blindly generate trading signals, but to help a portfolio or client-facing team understand which themes matter and how they may affect ETF or equity portfolio positioning.

La mia raccomandazione

Sì a **NLP + ML signal classification**, ma chiamalo:

market theme classification
macro regime classification
portfolio relevance scoring
research prioritization
ETF/equity portfolio implications

Non chiamarlo troppo:

alpha prediction, **news trading bot**, **automated investment signal**, perché per questo ruolo può sembrare meno CPM e troppo trading/quant research puro.

che vuol dire relevance scoring?

Relevance scoring vuol dire assegnare a ogni news/documento un **punteggio di rilevanza**, per decidere se vale la pena includerlo nel report oppure ignorarlo.

Nel tuo caso: il sistema legge tante news finanziarie, ma non tutte sono utili. Quindi dai uno score, per esempio da **0 a 1** o da **0 a 100**, che misura quanto quella notizia è importante per il tuo obiettivo.

Esempio pratico per QIS/ETF:

Una news tipo:

> "US CPI comes in above expectations; Treasury yields rise and Nasdaq futures fall."

Potrebbe ricevere:

```
Theme: Inflation / Rates
Asset class: Equities, Rates
Region: US
Affected portfolios: Growth equities, Nasdaq ETF, Quality factor ETF
Relevance score: 0.92
Reason: Direct impact on equity valuations, factor rotation and ETF positioning.
```

Mentre una news tipo:

> "Small regional company appoints new CFO."

Potrebbe ricevere:

```
Theme: Corporate news
Asset class: Single stock
Region: US
Affected portfolios: Low relevance for broad ETF portfolios
Relevance score: 0.18
Reason: Limited macro or portfolio-level impact.
```

In parole semplici:

> **Relevance scoring = filtro intelligente che decide quali notizie sono davvero importanti per il report.**

Può essere calcolato usando varie cose:

1. Similarità semantica
Quanto la news è vicina ai temi che ti interessano: ETF, equity markets, inflation, rates, factor investing, volatility, flows.

2. Presenza di entità importanti
Se compaiono Fed, ECB, CPI, rates, S&P 500, Nasdaq, sector ETFs, earnings, oil, geopolitics, ecc.

3. Impatto potenziale sul portafoglio
Una news che può cambiare view su equity, settori, fattori o rischio ha score più alto.

4. Fonte e urgenza
Una news da fonte affidabile e recente può pesare di più.

5. Magnitudine della sorpresa
Per macro data: CPI sopra attese, payrolls molto diversi dal consensus, taglio/rialzo inatteso, shock geopolitico.

Formula semplice da raccontare:

```
Relevance score =
0.35 × semantic similarity to investment themes
+ 0.25 × macro/market entity importance
+ 0.20 × expected portfolio impact
+ 0.10 × source quality
+ 0.10 × recency
```

Per colloquio puoi dirla così:

> By relevance scoring, I mean assigning each news item a numerical score based on how useful it is for the research objective. For example, if the objective is to produce an ETF/equity market brief, a CPI surprise, Fed communication or sector rotation story would receive a high score, while isolated company-specific news with no portfolio-level implication would receive a low score. This helps the system filter noise and prioritize the news that actually matters for market commentary and client-facing research.

e come lo usi. possiamo usare ML per classificare, ma relevacne scoring?

Thought for 9s

Sì, puoi usare **ML anche per relevance scoring**. Anzi, tecnicamente relevance scoring può essere visto come una **classification/regression/ranking problem**.

La distinzione è questa:

ML classification risponde a:

> "Di che tipo è questa news?"

Esempio:

> inflation / rates / earnings / geopolitical risk / sector rotation / ETF flows

Relevance scoring risponde a:

> "Quanto è importante questa news per il mio report, portafoglio o cliente?"

Esempio:

> score 0.91 = molto rilevante, includila nel brief
> score 0.22 = poco rilevante, ignorala

Quindi non sono alternative. Sono due layer diversi.

Come lo userei nella tua pipeline

Immagina che arrivino 500 news al giorno.

Prima fai **classification**:

```
News: "US CPI comes in above expectations, Treasury yields rise"

Theme: Inflation / Rates
Region: US
Asset class: Equities, Rates
Affected style: Growth, Quality, Momentum
Sentiment: Negative for long-duration equities
```

Poi fai **relevance scoring**:

```
Relevance score: 0.93
Reason: macro surprise, direct impact on rates, equity valuations and factor rotation
Action: include in daily ETF/equity market brief
```

Altra news:

```
News: "Small-cap company appoints new CFO"

Theme: Corporate news
Region: US
Asset class: Single stock
Sentiment: Neutral
Relevance score: 0.12
Action: ignore
```

La classification ti dice **cos'è**.
Il relevance score ti dice **se merita spazio nel report**.

Come costruirlo concretamente

All'inizio puoi farlo **senza ML**, con una formula rule-based. È più facile da spiegare in colloquio.

Per esempio:

```
Relevance score =
0.30 × similarity_to_target_themes
+ 0.25 × macro_entity_importance
+ 0.20 × portfolio_exposure_match
+ 0.15 × market_surprise_or_price_move
+ 0.10 × recency
```

Dove:

similarity_to_target_themes = quanto la news è vicina a temi come rates, inflation, equity markets, ETF flows, factor rotation.

macro_entity_importance = se cita Fed, ECB, CPI, payrolls, yields, S&P 500, Nasdaq, oil, credit spreads, ecc.

portfolio_exposure_match = quanto è rilevante per i portafogli/ETF che sto monitorando. Una news sui tassi è molto rilevante per growth/quality ETFs, magari meno per un ETF difensivo.

market_surprise_or_price_move = se la news è associata a un dato macro sopra/sotto consensus o a un movimento forte di mercato.

recency = una news di oggi pesa più di una news di tre giorni fa.

Dove entra il ML?

Il modo più pulito è questo:

1. Usi ML/NLP per classificare le news.
2. Crei feature strutturate.
3. Addestri un modello a stimare la probabilità che una news sia rilevante.
4. Quella probabilità diventa il relevance score.

Esempio:

```
Input features:
- theme = inflation
- region = US
- asset_class = equities/rates
- sentiment = negative
- mentions_Fed = yes
- mentions_CPI = yes
- semantic_similarity_to_ETF_themes = 0.88
- market_move_after_news = high
- source_quality = high
- novelty_score = 0.76

Output:
P(relevant) = 0.91
```

Quindi il relevance score può essere semplicemente:

```
relevance score = model.predict_proba(news_is_relevant)
```

Se hai label manuali tipo:

```
Relevant / Not relevant
```

puoi usare logistic regression, random forest, XGBoost, LightGBM, ecc.

Se invece hai ranking tipo:

```
News A più importante di News B
```

puoi usare un modello di **learning-to-rank**.

Versione realistica per il tuo progetto

Per non sembrare che stai overclaimando, io direi così:

> I first used a rule-based relevance score combining semantic similarity, macro entity importance, recency, source quality and portfolio exposure match. Then, as a possible extension, I would use manually reviewed research briefs as labels to train a supervised model that predicts whether a news item should be included in the final report. In that setup, the model output probability becomes the relevance score.

Questa è molto credibile.

Come lo racconti in colloquio

Puoi dire:

> The classification layer tells me what the news is about, while the relevance scoring layer tells me whether it is important enough to include in the research brief. For example, a CPI surprise would be classified as inflation/rates, but it would also receive a high relevance score because it directly affects equity valuations, factor rotation and ETF positioning. A company-specific news item with no portfolio-level implication may still be classified correctly, but it would receive a low relevance score and be filtered out.

Poi aggiungi:

> Initially I would implement relevance scoring with a transparent rule-based model using semantic similarity, entity importance, recency, novelty and portfolio exposure match. Once enough labelled examples are available, I would train a supervised classifier where the output probability represents the relevance score.

Questa è la risposta migliore: **classification + scoring + ranking + filtering**, senza far sembrare che l'LLM inventi segnali finanziari.

e ma come viene fatta relevance scoring? l'algoritmp

Thought for a couple of seconds

Bro, l'algoritmo più concreto e credibile è questo:

> **Relevance scoring = weighted scoring model**
> cioè trasformi ogni news in feature numeriche e poi fai una media pesata.

Non serve partire subito con ML complesso. Puoi costruirlo così.

---

1. Input

Ogni news entra così:

```
Title: "US CPI comes in above expectations as Treasury yields rise"
Source: Reuters
Date: today
Text: "US inflation surprised to the upside..."
```

Il tuo sistema deve decidere:

```
Quanto è rilevante questa news per un brief macro/equity/ETF?
```

---

2. Estrai feature

Per ogni news calcoli 5–6 score parziali.

A. Semantic similarity score

Misuri quanto la news è simile ai temi che ti interessano.

Temi target:

```
inflation
rates
central banks
equity markets
ETF flows
factor rotation
volatility
earnings revisions
geopolitical risk
sector rotation
```

Usi embeddings.

Esempio:

```python
news_embedding = embed(news_text)
theme_embedding = embed("inflation rates equity market ETF portfolio positioning")

similarity_score = cosine_similarity(news_embedding, theme_embedding)
```

Output:

```
similarity_score = 0.87
```

Vuol dire: questa news è molto vicina ai temi che mi interessano.

B. Entity importance score

Controlli se nella news compaiono entità importanti.

Esempio pesi:

```python
entity_weights = {
    "Fed": 1.0,
    "ECB": 1.0,
    "CPI": 0.9,
    "payrolls": 0.9,
    "Treasury yields": 0.8,
    "S&P 500": 0.8,
    "Nasdaq": 0.8,
    "oil": 0.6,
    "single company": 0.2
}
```

Se la news cita **CPI + Fed + Treasury yields**, ha entity score alto.

Esempio:

```
entity_importance_score = 0.90
```

C. Portfolio exposure match

Questa è la parte più bella per QIS/ETF.

Controlli se la news è rilevante per i portafogli o ETF che stai monitorando.

Esempio:

```
Portfolio/ETF exposure:
- US equities
- Quality factor
- Growth factor
- Low volatility
- Technology sector
```

News:

```
"CPI above expectations, yields rise"
```

Impatto:

```
Relevant to growth equities, quality, long-duration equities, Nasdaq exposure
```

Score:

```
portfolio_match_score = 0.85
```

Una news su una small cap random invece avrebbe:

```
portfolio_match_score = 0.10
```

D. Novelty score

Eviti di mettere 15 volte la stessa news.

Con embeddings confronti la news nuova con quelle già viste nelle ultime 24/48 ore.

```python
novelty_score = 1 - max_similarity_with_recent_news
```

Se la news è nuova:

```
novelty_score = 0.90
```

Se è duplicata:

```
novelty_score = 0.15
```

E. Recency score

Più è recente, più pesa.

Formula semplice:

```python
recency_score = exp(-hours_since_publication / 24)
```

Esempio:

```
news published 2 hours ago → recency_score = 0.92
news published 2 days ago → recency_score = 0.14
```

F. Source quality score

Dai più peso a fonti affidabili.

Esempio:

```python
source_score = {
    "Reuters": 1.0,
    "Bloomberg": 1.0,
    "Financial Times": 0.9,
    "central bank website": 1.0,
    "random blog": 0.3
}
```

---

3. Formula finale

Poi combini tutto:

```
Relevance Score =
0.30 × semantic_similarity
+ 0.25 × entity_importance
+ 0.20 × portfolio_match
+ 0.10 × novelty
+ 0.10 × recency
+ 0.05 × source_quality
```

Esempio concreto:

```
semantic_similarity = 0.87
entity_importance = 0.90
portfolio_match = 0.85
novelty = 0.75
recency = 0.95
source_quality = 1.00
```

Calcolo:

```
0.30×0.87 + 0.25×0.90 + 0.20×0.85 + 0.10×0.75 + 0.10×0.95 + 0.05×1.00

= 0.876
```

Quindi:

```
Relevance score = 0.88
```

Decisione:

```
Score > 0.75 → include in report
0.40 – 0.75 → maybe include / send to review
< 0.40 → ignore
```

---

4. Esempio completo

News:

```
"US CPI comes in above expectations, Treasury yields rise and Nasdaq futures fall."
```

Output del sistema:

```
Theme: Inflation / Rates
Region: US
Asset class: Equities, Rates
Affected exposure: Growth equities, Nasdaq, Quality factor
Sentiment: Negative for long-duration equities
Relevance score: 0.88
Action: Include in daily macro/equity brief
Reason: High macro importance, direct impact on yields, equity valuations and factor rotation.
```

Altra news:

```
"Small US company appoints new CFO."
```

Output:

```
Theme: Corporate news
Region: US
Asset class: Single stock
Affected exposure: None / limited
Sentiment: Neutral
Relevance score: 0.16
Action: Ignore
Reason: Low portfolio-level relevance for broad equity/ETF commentary.
```

Dove entra il ML?

Il primo algoritmo è **rule-based weighted scoring**. È trasparente e facile da spiegare.

Poi puoi evolverlo con ML così:

Prendi un dataset di news storiche e metti label manuali:

```
1 = included in research brief
0 = not included
```

Crei feature:

```
semantic_similarity
entity_importance
portfolio_match
novelty
recency
source_quality
sentiment_score
theme
region
asset_class
```

Poi alleni un modello:

```python
model = LogisticRegression()
model.fit(X_train, y_train)

relevance_score = model.predict_proba(X_new)[0, 1]
```

A quel punto:

```
Relevance score = probabilità che quella news debba essere inclusa nel report.
```

Quindi il ML non sostituisce la logica. Impara i pesi migliori dai dati.

Come lo dici in colloquio

> I would implement relevance scoring initially as a transparent weighted scoring model. For each news item, I calculate features such as semantic similarity to target investment themes, importance of extracted macro entities, portfolio exposure match, novelty versus previously retrieved news, recency and source quality. These features are normalized between 0 and 1 and combined into a final relevance score. For example, a CPI surprise mentioning Treasury yields and affecting growth equities would receive a high score and be included in the brief, while isolated company-specific news would be filtered out. As an extension, once enough labelled examples are available, I would train a supervised classifier where the predicted probability of inclusion becomes the relevance score.

Questa è una risposta solida perché mostra algoritmo, logica finanziaria e possibilità di ML senza sparare fuffa.

Developing hybrid LLM/ML research pipelines that transform financial news into structured data, using embeddings to identify emerging macro narratives. Built a multi-label classification framework mapping news clusters into macro state vectors across growth, inflation, monetary policy, liquidity, and risk sentiment to support macro monitoring and portfolio research.

Thought for 5s

Sì, è forte, però così com'è è **un po' troppo densa e "quant buzzword"**. Rischia che un recruiter CPM/ETF pensi: *ok bravo, ma cosa produce concretamente?*

Io la renderei più chiara e più collegata a **portfolio research / market commentary / client reporting**.

Developing hybrid LLM/ML research pipelines that convert financial news into structured macro signals for portfolio research. The workflow uses embeddings to cluster related news, identify emerging macro narratives, and map them through a multi-label classification framework into interpretable state variables across growth, inflation, monetary policy, liquidity and risk sentiment, supporting macro monitoring, ETF commentary and portfolio positioning analysis.

Questa è la versione migliore secondo me: mantiene LLM/ML, embeddings, macro narratives e classification, ma chiarisce **a cosa serve**.

Se vuoi una versione più "CV investment management" e meno tecnica:

Built an AI-assisted macro research pipeline using LLMs, embeddings and structured classification to transform financial news into concise market intelligence. The system clusters related articles, identifies emerging macro themes and tags them across growth, inflation, policy, liquidity and risk-sentiment dimensions to support portfolio research, ETF commentary and client-facing market briefs.

Questa forse è ancora più adatta al ruolo Goldman QIS ETF CPM, perché "client-facing market briefs" e "ETF commentary" sono molto pertinenti.

Occhio solo a una cosa: **"macro state vectors" è figo ma può sembrare troppo accademico**. Se lo usi, devi saperlo spiegare facile:

> By macro state vector, I mean a structured representation of the market environment. For each day or week, the system assigns scores to dimensions such as growth, inflation, monetary policy, liquidity and risk sentiment. For example, after a hawkish Fed meeting and stronger CPI data, the vector could show higher inflation pressure, tighter monetary policy and weaker risk sentiment. This makes qualitative news easier to compare over time and connect to portfolio commentary.

La versione più naturale da colloquio:

> The idea was to avoid using GenAI only for summarization. I wanted to turn news into structured data. So the pipeline first retrieves and clusters related articles using embeddings, then extracts the key themes, and finally classifies each news cluster across macro dimensions like growth, inflation, monetary policy, liquidity and risk sentiment. The output is not an automatic trading signal, but a structured macro dashboard that helps identify which narratives are becoming more important and how they may affect ETF positioning or client commentary.

Io nel CV userei la **seconda versione** se vuoi massimizzare fit col ruolo CPM/ETF.
Userei la **prima** se vuoi sembrare più quant/AI.